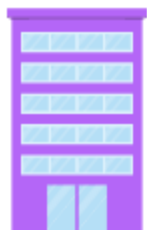


Winter-Dev Ants 제로.배트 PMP

Ants 팀장 정휘찬

CONTENTS

01



주 제

02



팀 목표

03



개인 목표

04



그라운드 룰

05



개발 일정

주제

04



주제 소개

- 제로시버트 모작 + 확장
- 2D 탐부 아포칼립스 슈팅 어드벤처

게임 장르

- 생존 (PVE)
- 배틀로얄 (PVP)

차별점

- 멀티플레이 위주 플레이

주제

04

신지훈

관심 분야: 웹
기술 스택: Java

서민지

관심 분야: 웹
기술 스택: Java, JS

설민우

관심 분야: 게임개발
기술 스택: C#, Unity

김성신

관심 분야: 게임개발
기술 스택: C#, Unity



선택이유

본인이 원하는 분야 성장 가능

- 로그인 관련 웹
- 게임 관련 서버들
- 게임 클라이언트

캠프 이후 발전 가능성

- 유사 기능을 가진 게임이 시장에 존재하지 않기 완성도를 높여 출시를 바라볼 수 있습니다

정휘찬

관심 분야: 게임개발
기술 스택: C++, Unreal

팀 목표

01

**서버 이해**

- 목 표 : 프레임워크를 익혀 클라이언트와 서버 구조를 이해한 뒤 설계 및 구현
- 선정 이유
 - 팀원의 대부분이 개발 경험이 적어서 서비스에 필요한 도구를 공부할 필요가 있다
 - 클라이언트 개발만 하던 팀원도, 서버와의 통신을 생각하며 설계 및 구현하여 아키텍처에 대한 이해
 - 서버 개발도 어떤 데이터를 어디서에서 처리할지 생각하고 설계 및 구현

**협업**

- 목 표 : 생각을 글로 표현하는 능력 성장 및 코드리뷰 문화 정착
- 선정 이유
 - 내가 작성한 문서를 팀원들과 피드백하면 문서 작성 능력 향상
 - 다른 언어, 프레임워크의 코드를 접하면서 새로운 기술에 대한 적응력 향상

팀 목표

01



출시

- 목표 : 게임 출시하기
- 선정 이유
 - 열정적인 개발 독려



개인 목표

현재

김성신

- 아키텍처 설계를 하지않고 개발을 먼저 한다음에 필요할 때 마다 자세한 계획 없이 막무가내로 새로운 스크립트를 작성
- 게임 개발을 끝까지 하지 않고 중요 기능만 개발을 하고 끝냄
- 네트워크 관련 실제 개발 경험 없음
- 문제 해결 경험 정리 안함

서민지

- 수업에 필요할 때(과제, 실습)에만 개발을 접하고 심화적 학습이나 프로젝트를 진행한 적이 없어 개발을 직업으로 삼을지 확신이 없음
- 실습보단 이론 위주의 수업으로 개발자의 역량을 확인 하지 못함

이후

김성신

- 멀티플레이 게임의 Client 와 Server 간의 구조(패킷 교환 방식)를 이해하고 게임 설정과 같은 게임 외적인 부분까지 구현해서 게임 개발의 전과정을 아는 상태에 이루기

서민지

- 개발 과정을 담은 글을 통해 개발에 대한 흥미, 적합성, 인내력 등을 주체적으로 평가할 수 있는 능력 갖기
- 설계 부터 구현, 협업을 수행하고 팀원들의 피드백을 통해 개발의 전 과정을 겪어 보며 진로에 대한 확신 갖기

개인 목표

현재

신지훈

- 아키텍처에 대한 이해가 부족하고 막상 개발을 시작하면
- 비슷한 기능이 다른 곳에 있어 효율적이지 않음
- 프로젝트에 어떤 점이 필요한지 잘 몰라 구글링을 하며 즉흥적인
- 개발을 하며 효율성을 떨어뜨림

정휘찬

- 클라이언트 서버 구조, IDCP 모델 이해 부족
- 프로젝트 경험이 적어 코딩에 자신감이 없음, 코딩 주저

이후

신지훈

- nest.js 프레임워크구조에서 정형화된 패턴 숙지 및 사용으로 효율성이 있게 작업하기.
- 사전에 철저한 조사를 통해 기능단위로 단기목표를 반복적으로 세우고 사전계획에 대한 수정횟수를 줄이며 개발하기.
- 하나의 프레임워크를 정확히 익혀서 서비스를 개발할 때 어려움이 없는 상태

정휘찬

- 서비스에 맞는 통신 구조 설계 능력, 멀티 쓰레드 프로그래밍 능력을 가진 상태
- IDCP 모델의 통신 구조, 패킷 흐름 이해
- 스레드로 나눌 때 성능적 이점 얻는 구조, 패턴 파악



개인 목표

현재

설민우

- 개발자끼리의 협업활동은 경험이 적어 코드들이 개인 중 심적임(협업에 원활하지 않음)
- 기존에 작업방식이 기능구현에 초점이 맞춰져 있어 기능이 완성되면 최적화 작업을 건너뛰고 완결 지어버림.
- 작업을 문서화 진행없이 머릿속에 있는 계획으로 작업을 진행하여 기존의 코드를 재차 수정하는 일이 잦았음.

이후

설민우

- 목표 : 코드를 가볍게 작성할 수 있고, 협업이 원활하게 가능한 상태 도달
- Profiler overview 등의 효율성을 판단할 수 있는 도구들을 이용해서 상황 진단, 최적화 작업할 수 있는 상태 도달.
- 다수의 인원들과 활동함에 따라 노션(Markdown)을 이용한 문서화 능력을 길러 의사소통에 문제가 없도록 하기.
- 핵심 기능 단위의 기획 및 설계 작업(UML이용) 후 구현함으로써 불필요한 재작업 최소화.

개인 목표

04

달성 과정

김성신

- 멀티플레이 게임 개발과정 이해
- 목표 설정 이유 :
- 개발
 - 유니티 C# 게임 서버 개발 공부
 - 본격적인 개발 시작 전 멀티플레이에 필요한 지식(어떤 지식) 공부 완료
 - 게임 개발 인강 수강(MMORPG 게임 개발 시리즈 구매완)
 - DB,
 - Network(네트워크 연결 및 패킷 교환 방식)
 - 유니티 연동
 - 멀티 쓰레드
 - protocol buffer 사용한 서버간 연동 이해
- 협업
 - 팀관리
 - Github
 - branch 규칙 설정
 - commit 규칙
 - git project or asna 와 같은 툴 사용해서 일정 관리
- 문서화
 - 코드 관련 문서화
 - usecase, 시퀀스 다이어그램, 클래스 다이어그램 등을 사용해 팀원들도 쉽게 코드 이해
 - 회의록, 그라운드를 팀 관리 측면 문서화

개인 목표

04

달성 과정

서민지

1. 개발 일기 작성

- 당일 작업한 내용과 함께 감정, 생각 등을 함께 기록하기
- 진행 중 부족한 점이 있었다면, 해결하기 위해 어떤 노력을 했는지, 해결할때의 감정등을 숨김없이 기록하기
- 성취감과 같은 긍정적인 감정을 느꼈다면, 어떤 상황에서 도출된 것인지 기록하기
- 최종적으로 캠프가 끝났을때 일기를 돌아보며 성장 여부를 판단하고 개발에 대한 생각 정하기

2. 개발의 전 과정 밟기

- 베끼기만 하는 코딩이 아닌 반복적인 이해를 통해 내것으로 만들려는 노력하기
- 1시간 30분 코딩/ 15분 휴식의 규칙적인 패턴과 같이 최대한 몰두할 수 있는 시간을 찾고, 코딩 몰두 시간 점차적으로 늘리기
- 오류 발생시 체계적이고 자발적으로 해결하려는 자세 가지기
 1. Debug.Log()를 활용하여 정확한 오류 위치 찾기
 2. 구글링을 통한 직접 해결
 3. 팀원에게 도움 요청

개인 목표

04

달성 과정

신지훈

- 아키텍처에 대한 이해가 부족하고 막상 개발을 시작하면 비슷한 기능들이 흩어져있어 코드의 응집도를 떨어뜨림
 - 프로젝트에 어떤 기능이 필요한지 사전조사를 하지 않고 구글링을 하며 즉흥적인 개발을 하며 불필요한 반복작업이 생김
- 목표
1. nest.js 프레임워크구조에서 정형화된 패턴 숙지하고 이 구조를 지키며 유지보수에 적합한 구조로 개발하기
 2. 개발 전에 철저한 조사를 통해 기능단위로 단기목표와 계획을 반복적으로 세우고 최대한 계획을 수정하지 않고 개발하는 상태
 3. 하나의 프레임워크를 정확히 익혀서 서비스를 개발할 때 어려움이 없는 상태
- 1-1 개발에 시작하기 전에 적절한 프레임워크(nest.js) 구조와 흐름 이해하기
 - 1-2 각각의 클래스, 변수, 모듈 명의 형식을 통합해 개발할 때 효율성, 가독성 높이기
 - 1-3 느슨한 결합을 적극적으로 활용하고 재사용, 유지보수에 적합한 구조 갖추기(solid)
- 2-1 개발 전 프로젝트에 어떤 요구사항이 있는지 자세하게 살피고 기록하기
 - 2-2 하나의 요구사항을 단기목표로 설정한 뒤 해당기능에 필요한 라이브러리,기술 조사하기
 - 2-3 라이브러리와 기술을 바탕으로 해당 요구사항을 어떻게 구현할지 생각하고 기록하기
 - 2-4 팀 프로젝트에 맞춰 입력받아야 하는 데이터와 리턴해줘야하는 데이터를 공유하는 등의 사전에 통합해야하는 부분 잘 확인하여 반복작업 줄이기
 - 2-5 최대한 계획을 수정하지 않고 개발할 수 있도록 반복적으로 연습하기
- 3-1 위의 과정을 통해 하나의 기술에 깊이있게 공부하는 계기, 기반을 다지기 (T자형 지식 가지기)



개인 목표

04

달성 과정

설민우

Profiler overview 등의 효율성 을 판단할 수 있는 도구들을 이용해서 상황 진단, 최적화 작업할 수 있는 상태 도달. + 핵심 기능 단위의 기획 및 설계 작업(UML이용) 후 구현함으로써 불필요한 재작업 최소화.

아래와 같은 구조를 반복하여 작업

- 1)코딩 전 사전에 핵심 기능의 연계성 및 세부 기능들을 파악하여 대략적인 작동 WorkFlow 그려보기.
- 2)이를 기반으로 세부적인 계획서 작성하기(필요 컴포넌트 및 스크립트 구현방식, 필요 변수 및 함수 구상 - UML, TreeDiagram 이용),
- 3) 작업된 문서를 바탕으로 실작업 실시.
- 4) 기능 구현 이후 최적화 작업(Profiler overview 와 같은 도구 이용)
- 5) 최적화 작업 이후 기존에서 어떤 변경 사항이 있는지 자체 평가 실시 후 기록 작성.

(이때 어떠한 이유에서 변경되었는지, 다음에 설계시 참고할 점이 어떤것이 있는지 확실하게 기록 후 차후 작업시 반영.)

자체 평가내역 : 기획 & 설계와 실제 구현사이 차이점 및 이유 / 최적화 작업시 변경 내용 및 정도

개인 목표

04

달성 과정

설민우

다수의 인원들과 활동함에 따라 노션(Markdown)을 이용한 문서화 능력을 길러 의사소통에 문제가 없도록 하기.

1) 이해하기 명확한 변수 및 함수명 사용으로 참조에 있어 불편함이 없도록 준비하기.(C# 권장 코딩 컨벤션 이용)

-> 의사소통에 오해나 추가적인 설명이 필요할 시, 원인을 분석하여 작업방식 업데이트.

2) 기능 구현시 요약된 해설(ReadMe)문서를 작성하여 WorkFlow를 쉽게 이해할수 있도록 만들기.

-> 협업 필요시 이해하기 어려운 부분이 파악되면 이를 수정하고, 다른 부분에도 비슷한 내역으로 최신화 할 부분이 있는지 검토하기.

3) 문제 해결(트러블슈팅) 이후 노션을 통해 이와 관련된 자료 정리 및 공유.

-> 작업한 결과 및 발생한 문제 해결 방법을 일일단위로 노션을 통해 공유하여 현재 작업 상황 및 문제 해결 방식 공유하기.

한 주가 끝난 시점에서 팀원들에게 피드백을 받아 개선 방향 확인 후 작성 방식 수정.

개인 목표

04

달성 과정

정휘찬

1. I/OCP 모델을 사용한 서버 구조 이해

- 서로 다른 사람이 작성한 코드 분석하여 학습 (구조 분석, 흐름 이해)
- Inflearn 강의 에서 사용한 코드(C++ 게임 서버)
- 유튜브 '최흥배' 채널에서 소개한 코드(C++ I/OCP MOC서버)
- 캠프장님께 받은 책 (온라인 게임 서버 프로그래밍)
- 차이점과 장단점 분석
- 학습 내용 개인 프로젝트 (URL shortener)에 적용해 구조 변경
- 변경 전 후 구조, 개선된 점 문서로 정리
- 프로젝트 후에도 코드 리뷰 및 질문, 검색, 직접 설계 및 구현을 통해 더 나은 방향 지속적 탐구

2. 서비스에 적합한 서버 구조 설계

- 필요한 기능을 정리 하여 유스케이스 다이어그램 작성
- 시퀀스 다이어그램 작성 후 객체를 더 작은 객체로 쪼개어 다시 작성 (흐름 구체적으로 파악)
 - ex) 클라이언트 - (게임 서버) -> 클라이언트 - (리스너 - 패킷 분류기 - 패킷 처리기)

3. 안전한 멀티 스레드 활용

멀티 스레드 환경 인지 후 클래스 설계

- 어떤 변수를 공유해야 할지
- 경쟁 조건이 발생하는지
- 동기화 문제가 발생하는지

서버의 플로우 차트 작성 후 병렬화 가능성 분석 및 평가

그라운드 룰

01



코어 타임

- 평일 오후 2~5시



정기 회의

- 회의주제
 - 주간 목표 설정 및 평가
 - 개발 진척도 관리
 - 협력이 필요한 부분 회의
 - 팀원 이슈 확인
- 회의 일시
 - 매주 금요일 1시



스크럼 회의

- 회의 주제(시작 스크럼)
 - 당일 업무 정리
 - 협력 리마인드
 - 개발 이슈 확인
- 회의 일시
 - 매일 코어타임(2~5시) 전
- 회의주제(종료 스크럼)
 - 진척도 확인
 - 이슈 발생 시 공유
- 회의 일시
 - 매일 코어타임(2~5시) 이후

그라운드 룰

01

역할 분담



정휘찬

- 팀장
 - 정기회의 주관
 - 캠프 진행 관련 연락



김성신

- Github Maintainer
 - branch 관리
 - 규칙 준수 유무 확인
- 서기
 - 매 회의 시 회의록 작성



설민우

- ToDoList 관리자
 - 일정 관리
 - 일정 관리 툴 최신화



서민지

- Scrum Master
 - 회의 진행
 - 주간 업무 설정
 - 진척 관리



신지훈

그라운드 룰

01

업무 분담



로비 UI(서민지)



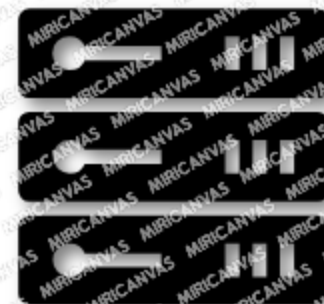
Client(설민우)



인증서버
(신지훈)



로비 서버
(신지훈)



매칭 서버
(김성신)



게임 서버
(정휘찬)

그라운드 룰

01

이슈 해결



1차

- 노션 및 슬랙을 통해 팀원에게 상황 보고 후 개인 진행
- 팀원의 간접적 도움
- GitHub SlackBot 활용



2차

- 팀원의 직접적인 도움 요청



3차

- 멘토 및 캠프장님께 도움 요청

그라운드 룰

04

위기 관리

위험

4개의 서버를 사용하기 때문에 클라이언트와 통신에 문제가 생길 수 있다

개발 경험, 사전 지식 부족으로 계획의 현실성 판단 불가



해결 방안

하향식 통합으로 연결 상태 먼저 점검
지속적인 통합테스트

철저한 자료 조사 및 적극적으로 질문

그라운드 룰

04

코드 컨벤션

Camel

각 단어의 첫글자 대문자 표기하되
첫 단어의 첫글자는 소문자 표기

- 변수명

Pascal

각 단어의 첫글자 대문자 표기

- 함수명
- 클래스명

제어문

제어문 생성 시 제어문과 괄호 사이
에 공백
중괄호 내리기

```
if ()  
{  
  //ToDo  
}
```

private 멤버 변수

private 멤버 변수 앞에 ‘_’

주석

코드는 줄의 끝이 아닌 별도의 줄에 주석을 배치
주석 텍스트는 대문자로 시작
주석 텍스트 끝에는 마침표

그라운드 룰

04

commit 컨벤션

(a)

기능 : 기능에 관한 작업

문서 : 문서 수정 시

양식 : 기능에 영향없는 작업(오타 수정, 띄어쓰기)

(b)

추가

수정

삭제

[(a) (b)]

예시:

[기능 추가] 게임서버에서 매칭 서버와의 통신을 위한 기능 추가, X 클래스 생성, Y 클래스

생성

기타

[리팩토링] : 리팩토링 시

[테스트] : 테스트 시

개발 일정

간트 차트

담당	업무	1월 3주차	1월 4주차	1월 5주차/ 2월 1주차	2월 2주차	2월 3주차
신민우	데이터 베이스					
	플레이어 조작					
	FOV					
	공격 기능					
	체력					
	장비					
	맵 구성					
	적 AI					
	포톤서버 PVE					
	디버깅, 빌드					
민지	UI 배치도 작성					
	로그인창					
	채팅창					
	설정창					
	캐릭터창(정보, 스킬)					
	캐릭터창(인벤토리)					
	상점					
지훈	ERD 구축, 구현					
	로그인 서버					
	상점 API 구현					
	개인 인벤토리					
	연 동					
	테스트					
	배포 준비					
	추가 서비스					

담당	업무	1월 3주차	1월 4주차	1월 5주차/ 2월 1주차	2월 2주차	2월 3주차
정휘찬	기본 네트워크 구조					
	매칭 서버 패킷 처리					
	매칭 서버 연결 테스트					
	룸 생성 및 참가					
	게임 패킷 처리 및 기능					
	게임 접속 종료 패킷					
김성신	매칭 요청					
	대기방 확인					
	대기방 생성					
	대기방 참가					
	게임 서버 연결					
	매칭 알고리즘					
	게임 설정					
전체	유니티 패킷 연결					
	통합 테스트					

